Component Showcase

Living documentation for homework-template

Student Name · Student ID

COURSE 101 · Course Name

Not applicable

Instructor Name · Collaborators: None

1 Cheat sheet

Everything this template offers, on one page. Each entry names the environment or command that produces it; the sections that follow show them in use.

Structure

`problem` ..... a numbered problem

`subproblems` ..... (a), (b), (c)

`solution` ..... worked reasoning

`answer` ..... the final result

`\DocFolder` ..... include a folder

Callouts

`callout` ..... neutral note

`tip` ..... what to remember

`warn` ..... a caveat

`caution` ..... a mistake to avoid

Mathematics

`theorem` `lemma` `proposition`

`corollary` `definition` `remark`

`proof` ..... ends with a tombstone

Code

`codebox[lang]{file}` ..... titled plate

`codeplain[lang]` ..... no title bar

`\inputcode` ..... a real file

`terminal` ..... shell transcript

`\code` `\filepath` ..... inline

Figures

`\HomeworkFigure` ..... an image

`\HomeworkDiagram` ..... a D2 diagram

Marks

✓ `\rzcheck` ✗ `\rzcross`

⚠ `\rzwarnglyph` ⓘ `\rzinfoglyph`

Two rules Write one problem per file in `fragments/problems/` and number with room to grow — `010`, `020`, `030`. Everything in that folder is included in name order, so a new problem is a new

file and `homework.tex` is never edited.

2 Problems and solutions

Problem 1: A numbered problem

12 points

Problem statements may contain prose, displayed mathematics, lists, and references. The optional argument records the available points.

(a) The first subproblem.

(b) The second subproblem.

Solution Solutions use a distinct breakable panel and may span multiple pages.

✓ Final answer Use an answer box when the final result should be easy to locate.

Problem 2: A problem without points

The points argument is optional, but the descriptive title is required.

3 Callouts

Four boxes for interrupting the argument. Each carries an icon and a word as well as a colour, so a reader in greyscale still gets which kind it is. The optional argument replaces the label.

Note Neutral context. The default label comes from the environment name.

Takeaway The one sentence you want the grader to remember.

⚠ Watch the bound The recurrence only closes for $n \geq 1$; the base case has to be checked separately.

× A common mistake Do not assume the loop terminates because it does on your inputs.

Custom label All four take an optional argument. Giving one turns the label into the sentence it introduces, rather than a generic banner.

4 Mathematics

Theorem 1 (Geometric series)

For $r \neq 1$ and every integer $n \geq 0$,

$$\sum_{k=0}^{n} r^k = \frac{1 - r^{n+1}}{1 - r}.$$

Proof. Multiply the sum by $1 - r$ and cancel adjacent terms. □

The template includes theorem, lemma, proposition, corollary, definition, and remark environments. Equations, theorems, figures, and tables can be referenced with `\cref`.

5 Images and diagrams

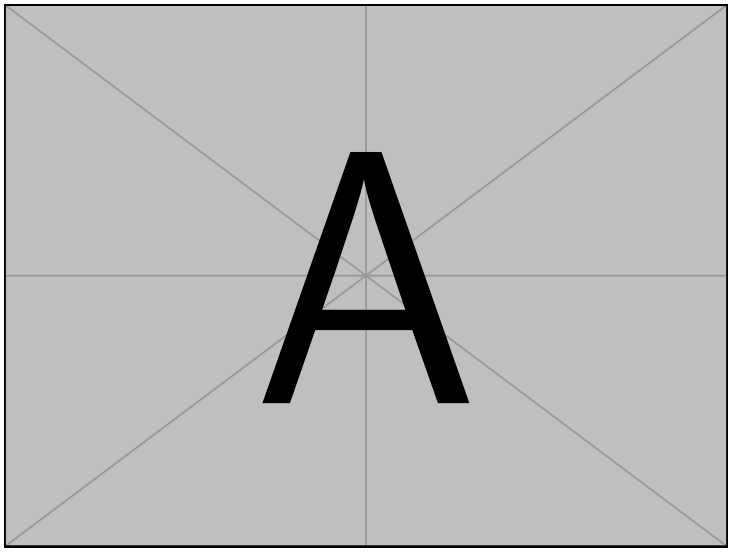

Figure 1: A sample raster-compatible figure included by the TeX distribution.

Every figure takes a visual description as its fourth argument, separate from the caption. The caption is a label for a reader who can see the figure; the description is the figure, in words, for one who cannot. Figure 1 carries both.

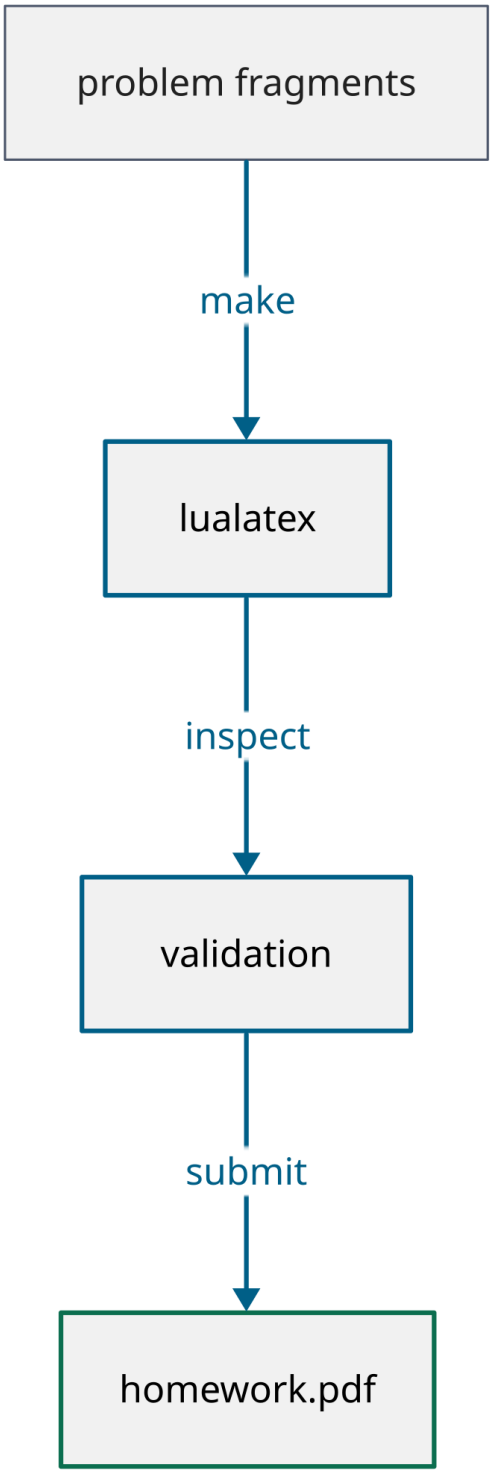

Figure 2: The source-to-submission workflow, drawn in D2.

Diagrams are written in D2 under `assets/diagrams/` and compiled to PDF by `make diagrams`. The generated PDFs stay committed, so editing a diagram needs D2 installed but compiling the document does not.

A diagram needs its description more than a photograph does, and here for two reasons rather than one.

The first is that the labels are not in the text layer at all. The pipeline goes through PostScript on its way to PDF, and that hop outlines the glyphs, so `pdftotext` returns nothing for a diagram. That is a deliberate trade: the PostScript route produces a byte-identical PDF on every run, which is what lets the build assert that a committed diagram still matches its source, and the direct route keeps the text but changes bytes between runs.

The second reason would hold even if the words were there. They would arrive as a loose sequence in drawing order, with no edges, no direction and no shape. Extractable text is not a description.

6 Code, tables, and citations

binary_search.py

```
def binary_search(values, target):
    low, high = 0, len(values) - 1
    while low <= high:
        middle = (low + high) // 2
        if values[middle] == target:
            return middle
        if values[middle] < target:
            low = middle + 1
        else:
            high = middle - 1
    return None
```

Table 1: Asymptotic costs of common search operations.

Method	Prerequisite	Worst case
Linear search	None	$\Theta(n)$
Binary search	Sorted input	$\Theta(\log n)$
Hash lookup	Suitable hash table	$\Theta(n)$

Table 1 uses spacing rather than decorative rules or vertical separators. Bibliographic citations use BibTeX and Natbib; for example, see Knuth (1984).

6.1 Other languages

The optional argument is the Pygments lexer, so any language Pygments knows is available. The signature is the slides project's on purpose: a fragment written for one template reads the same in the other.

sum_array.c

```c
size_t sum_array(const int *values, size_t n) {
    size_t total = 0;
    for (size_t i = 0; i < n; i++) {
        total += values[i];  /* no overflow check, on purpose */
    }
    return total;
}
```

Fib.hs

```haskell
fib :: Int -> Integer
fib n = fibs !! n
  where fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
```

6.2 Without a title bar

`codeplain` drops the tab and the line numbers, for an excerpt short enough that neither earns its space.

```sql
SELECT course, AVG(grade) AS mean
  FROM submissions
 GROUP BY course
HAVING COUNT(*) > 5;
```

6.3 Inline

Set an identifier with `\code`, as in `binary_search(values, target)`, and a path with `\filepath`, as in `fragments/problems/010-example.tex`. Both stay in Ubuntu Mono, so an identifier reads the same inline as it does inside a plate.

7 Terminal transcripts

When the answer is what a command printed, quote the session rather than paraphrasing it. The body is verbatim, so nothing inside needs escaping except the arguments of the macros themselves.

>_zsh

```
→ arch ~/homework make check
>> homework: LaTeX pass 1
homework.pdf: US Letter, tagged
✓ all 42 pairs meet WCAG 2.1 AA
✗ arch ~/homework make submit
✗ permission denied
⚠ 2 problems still marked TODO
```

The prompt follows the `ginger` zsh theme: a green arrow, then the host, then the directory. `\cmdfail` turns the arrow red *and* changes it to a cross, so a failed command is not distinguished by colour alone. `\TermHost` and `\TermPath` set the two labels, and both read their argument verbatim. That matters: a shell path is made of the characters LaTeX reserves, so `\TermPath{~/src/my_app}` is typed the way the shell shows it rather than escaped. The path defaults to `~`; pass an empty argument to drop it.

References

Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984. doi: 10.1093/comjnl/27.2.97.